Resume Screening and Scoring System: A Hybrid Approach Combining Rule-Based Filtering and Semantic Similarity Analysis

Group 33

Mahalaxmi, Aabha, Shreehitha
Department of Computer Science and Engineering
Indian Institute of Technology Jodhpur
Jodhpur, Rajasthan, India

***Abstract*—The hiring process in many organizations involves manually reviewing a large number of resumes, which is often time-consuming, inconsistent, and susceptible to bias. Existing approaches usually depend on keyword matching, which lacks contextual understanding, or use complex systems that are not transparent. This paper presents a Resume Screening and Scoring System that aims to balance efficiency and accuracy in the recruitment process. The system combines rule-based eligibility filtering with semantic similarity analysis using embeddings to match resumes with the corresponding job descriptions. It is developed using an incremental sprint-based development methodology and consists of key modules such as resume parsing, eligibility checking, embedding-based scoring, and ranked result visualization. The proposed system aims to reduce the manual effort while promoting fairness and transparency in the hiring process.**

***Index Terms*—Resume screening, semantic similarity, embeddings, automated hiring, rule-based filtering, NLP**

I. Problem Statement

A. Objective

The objective of the Resume Screening and Scoring System is to design a solution that makes the hiring process more efficient and fair. Instead of depending only on traditional keyword matching, the proposed system integrates structured eligibility rules with semantic similarity analysis. The system evaluates how closely a candidate's resume matches a job description.

The system models the key steps of the resume evaluation process. This includes text extraction, eligibility filtering, similarity scoring, ranking, and explanation of results. By combining rule-based filters, such as minimum qualifications or skill thresholds, with embedding-based semantic comparisons, the system captures both clear requirements and contextual meaning. This hybrid approach ensures that resumes are evaluated not only for required qualifications but also for meaningful alignment with job expectations. The overall aim is to reduce manual screening efforts while maintaining fairness, consistency, and transparency in candidate evaluation.

B. Motivation

The recruitment process is facing challenges due to the rapid increase in job applications. Recruiters often spend a lot of time reviewing resumes for a single hiring decision, yet only a small number of applicants reach the interview stage. This manual process is time-consuming and may lead to bias and inconsistent evaluation. As a result, qualified candidates might be missed.

Traditional Applicant Tracking Systems (ATS) mostly use keyword matching and Boolean search methods. Although these methods are computationally efficient, they fail to capture contextual meaning. For example, a resume mentioning "machine learning" may not connect with a job description asking for "artificial intelligence," despite the conceptual overlap between these skills. These limitations highlight the need for better text analysis techniques.

Recent advancements in natural language processing, specifically transformer-based language models and semantic embeddings, provide a chance to overcome these issues. These techniques allow systems to identify contextual similarity and measure relevance beyond just exact keyword matches.

Motivated by these developments, the proposed system takes a hybrid approach, combining rule-based eligibility filtering with embedding-based similarity scoring. This ensures that essential requirements are met while enabling meaningful comparisons between resumes and job descriptions. The system also emphasizes explainability, providing clear breakdowns of scores so recruiters can see why a particular candidate was ranked higher or lower. The goal is to develop a practical, scalable solution based on academic principles, suitable for both experimental evaluation and small-scale real-world use.

II. Review of Existing Technologies and Solutions

Before designing our Resume Screening and Scoring System, we studied how resume screening is handled in real companies and in research. This helped us understand what existing systems do well and where improvements are needed.

A. Existing Resume Screening Approaches

1) Commercial ATS Platforms: Many companies use Applicant Tracking Systems (ATS) to manage hiring. These systems help recruiters upload job descriptions, collect resumes, and manage interviews. However, most ATS platforms mainly rely on keyword matching. They check whether specific words from the job description appear in the resume.

Advantages:

- Easy-to-use dashboards for HR professionals
- Integration with job portals and LinkedIn
- Automated interview scheduling
- Built-in reporting and compliance tracking

Limitations:

- High yearly subscription cost, which may not be affordable for small companies
- Lack of clarity on how resumes are ranked
- Limited customization options
- Heavy dependence on exact keywords

For example, if the job description says "Machine Learning" and the resume says "ML," the system may fail to identify the match. This shows that many commercial systems are strong in workflow management but limited in semantic understanding.

2) Academic and Research Approaches: In research, resume screening methods generally fall into three categories:

Rule-based systems – Assign scores using predefined conditions. They are transparent but not flexible.

- *Pros:* Simple to implement, explainable scoring
- *Cons:* Requires manual rule creation, cannot handle variations in skill representation

Machine learning models – Learn patterns from past data. They improve accuracy but are harder to explain.

- *Pros:* Automatic pattern learning, better than rule-based systems
- *Cons:* Requires large labeled datasets, reduced transparency, potential bias propagation

Deep learning models – Use embeddings to understand context. They give better matching but increase computational complexity.

- *Pros:* Captures semantic meaning, handles synonyms and paraphrasing
- *Cons:* Computationally expensive, harder to explain, increased system complexity

From our review, we observed that no single approach perfectly balances accuracy, explainability, and system simplicity.

B. Resume Parsing and Similarity Methods

Resume screening first requires extracting structured information from resumes. We considered:

Pattern-based extraction

- *Pros:* Simple, works for clean resumes
- *Cons:* Fragile, fails with format changes

NLP-based extraction

- *Pros:* More flexible, handles varied formats
- *Cons:* Requires more processing power

External parsing APIs

- *Pros:* Highly accurate
- *Cons:* Costly, introduces external dependency

For similarity scoring, we analyzed:

TF-IDF

- *Pros:* Fast, simple
- *Cons:* Keyword-based, lacks semantic understanding

Word embeddings (Word2Vec, GloVe)

- *Pros:* Better semantic relations
- *Cons:* Context loss when combining word vectors

Sentence-level embeddings (SBERT)

- *Pros:* Strong contextual understanding
- *Cons:* Higher computational requirements

Based on this study, we chose a hybrid approach that combines eligibility filtering with embedding-based similarity, which also supports explainability.

III. Solution Trajectory

Our system evolved across multiple development stages, clearly reflected in SRS Version 2.0 and UML diagrams.

A. Evolution of System Design

1) Initial Concept: At the beginning, the idea was simple:

$$\text{Upload Resume} \rightarrow \text{Extract Text} \rightarrow \text{Generate Score}$$

However, after refining the requirements, we introduced structured modules.

2) Final Architecture (As per UML Diagrams): **Use Case Diagram:** The Recruiter can:

- Set eligibility criteria (CGPA, mandatory skills)
- Upload job descriptions
- Upload resumes
- View ranked results
- View rejection reasons

Sequence Diagram: The workflow follows:

1) Upload JD and resumes
2) Extract resume text
3) Fetch eligibility criteria
4) Apply CGPA and skill filters
5) Generate embeddings
6) Calculate similarity
7) Rank eligible resumes
8) Display ranked results and rejection reasons

Class Diagram: Key classes include: Resume, JobDescription, FilterEngine, EmbeddingEngine, Score, Recruiter. This shows clear separation of filtering and embedding logic, supporting maintainability.

Component Diagram: The system consists of:

- Web Interface
- Text Extraction Module
- Eligibility Filter Module
- Embedding & Ranking Module
- Data Storage

This modular structure improves scalability and clarity.

B. Development Approach and Sprint Planning

We followed an incremental sprint-based development model. The development was divided into 8 structured sprints over an 8-week timeline. Table I shows the detailed sprint plan with timelines and deliverables.

Each sprint improved the system gradually and aligned implementation with updated UML diagrams.

C. Scrum Roles and Alpha Managers

Based on the SRS:

Alpha Manager – Documentation & Design Lead

- **Sprints:** 1, 8
- **Role:** Finalize SRS, maintain UML consistency, handle traceability and final documentation

Alpha Manager – Frontend & User Interface

- **Sprints:** 2, 7
- **Role:** Develop Web UI, resume upload interface, result display, and usability improvements

Alpha Manager – Backend & Evaluation Logic

- **Sprints:** 3, 4, 5, 6
- **Role:** Resume parsing, eligibility filtering, scoring, ranking, and database interaction

This role distribution ensured clear ownership of modules.

Project Management Tool: The team plans to use Trello for managing project tasks and sprints. Trello is chosen for its simple, visual organization of tasks using boards and cards, which supports sprint-based development.

Using Alpha thinking (Opportunity, Stakeholders, Requirements, Software System, Team, Work), we tracked project maturity from initial idea to final system.

D. Technology Stack Selection

To support the modules shown in the component and class diagrams, we selected the following technologies:

Backend:

- Python (FastAPI) – Faster and more scalable than traditional Flask
- REST-based architecture

NLP and Processing:

- spaCy – For resume text extraction
- Sentence-Transformers (MiniLM model) – For efficient semantic embeddings
- Scikit-learn – For similarity computation and ranking logic

Database:

- PostgreSQL (Cloud-hosted) – Reliable and scalable for storing resumes, scores, and eligibility criteria

Frontend:

- React.js – Clean UI and dynamic result display
- Bootstrap / Tailwind CSS – UI styling

Deployment (Recommended):

- Docker for containerization
- Cloud hosting (AWS / Azure student tier)

This stack supports:

- Performance requirement (response within 3 seconds)
- Scalability
- Accuracy through embeddings
- Security through server-side processing

E. Development Evolution Table

Table II highlights what we incorporated or removed across development iterations:

F. Design and Implementation Conventions

The system follows consistent design and coding templates as specified in SRS Version 2.0:

- Class names follow PascalCase (e.g., ResumeService)
- Function names follow camelCase (e.g., generateEmbeddings)
- REST APIs use noun-based endpoints
- Single-line comments use // and multi-line comments use /* */
- UML diagrams follow standard UML 2.x notation

G. Summary of Work Progress

The UML diagrams included in our SRS represent the system in its final and current state at the time of submission (04 Feb 2026). Our iterative development approach ensured that:

- Each sprint refined the architecture
- Identified tech-stack components were validated
- Modular design principles were maintained
- Requirements were continuously aligned with implementation

IV. Results and Discussion

A. Design Approach Evaluation

During the initial design phase, we considered different approaches for automated resume screening.

1) Keyword-Based Matching (Initial Consideration): The first approach examined was a keyword-based matching system from classical information retrieval methods [1]. This method focuses on finding exact matches between job description keywords and resume content.

Theoretical Strengths:

- Simple to implement
- Fast processing
- Effective when requirements are clearly stated

Identified Limitations:

- Cannot capture semantic similarity (e.g., "Machine Learning" vs "ML")
- Sensitive to resume length and keyword frequency
- Lacks contextual understanding

Research in resume classification and text similarity [3] [4] shows that keyword-only methods often struggle with variations in language usage. Therefore, depending only on keyword matching would not be sufficient.

TABLE I
SPRINT-BASED DEVELOPMENT PLAN WITH TIMELINE

Sprint	Timeline	Activities	Deliverabl
Sprint 1 • Identify system modules • Prepare initial UML diagrams **Rationale:** Clear requirements and architecture are essential before implementation	Week 1 **Deliverable:** SRS document, UML diagrams (Use Case, Activity)	• Finalize SRS Version 2.0	
Sprint 2 • Set up backend project structure • Configure database connection **Rationale:** Foundation for user interaction and server-side processing	Week 2 **Deliverable:** Frontend skeleton, backend setup	• Create basic Web UI layout	
Sprint 3 • Validate file formats and size • Store files and metadata **Rationale:** Resume data required before analysis can begin	Week 3 **Deliverable:** Upload module	• Implement resume and JD upload	
Sprint 4 • Identify skills, education, experience • Store extracted data **Rationale:** Structured data necessary for eligibility checking	Week 4 **Deliverable:** Text extraction module	• Extract text from resumes	
Sprint 5 • Fetch eligibility criteria • Identify eligible and rejected resumes **Rationale:** Early filtering improves efficiency	Week 5 **Deliverable:** Eligibility filter module	• Implement CGPA and skill filters	
Sprint 6 • Compute semantic similarity • Rank eligible resumes **Rationale:** Core intelligence of the system	Week 6 **Deliverable:** Embedding and ranking module	• Generate resume and JD embeddings	
Sprint 7 • Show rejection reasons • Improve UI usability **Rationale:** User-facing outputs must be clear and interpretable	Week 7 **Deliverable:** Result presentation module	• Display ranked resumes	
Sprint 8 • Optimize performance • Finalize UML and documentation **Rationale:** Ensures reliability and readiness for evaluation	Week 8 **Deliverable:** Tested system, final documentation	• Functional and non-functional testing	

TABLE II
SYSTEM EVOLUTION ACROSS VERSIONS

Aspect	Changes
Parsing Method	Removed: External API dependency Added: In-house spaCy-based extraction
Scoring Approach	Removed: Pure keyword matching Added: Hybrid (rules + embeddings)
Architecture	Removed: Monolithic structure Added: Modular component design
Explainability	Added: Rejection reason display
Tech Stack	Changed: Flask $\rightarrow$ FastAPI for better performance
Database	Added: PostgreSQL for scalability

2) Semantic Similarity Approach (Alternative Consideration): To address these contextual limitations, we explored transformer-based semantic embeddings like Sentence-BERT [2]. These models aim to capture meaning instead of just matching specific words.

Theoretical Advantages:

- Can recognize paraphrased skills
- Better at handling synonyms
- Improved contextual alignment between resumes and job descriptions

However, further analysis revealed important limitations:

- Semantic similarity alone does not enforce structured eligibility rules (e.g., minimum CGPA, required certifications)
- Similarity scores do not clearly explain reject decisions

These considerations led to re-evaluation.

B. Architectural Refinement: Hybrid Model Selection

Based on theoretical analysis and literature review [1] [2] [3] [4], we arrived at a hybrid architecture that combines:

- Rule-based eligibility filtering
- Semantic similarity ranking

The rule-based element ensures validation of mandatory requirements. The semantic element captures contextual relevance. This combination balances clarity with contextual understanding, meeting the practical needs of recruitment systems where transparency is crucial.

C. Identified Constraints and Design Considerations

Even at the design stage, several constraints were identified:

1) Resume Format Variability: Resumes can vary in structure, layout, and terminology. This impacts how reliably we can extract text and requires flexible parsing methods.

2) Ambiguity in Skill Representation: Candidates might describe similar experiences in different ways. Context-aware processing is necessary to minimize mismatches caused by language differences.

3) Explainability Requirement: Recruitment systems need to provide clear reasoning. Therefore, architecture includes ways to explain decisions.

4) Data Availability: Large labelled datasets were not available of resumes during development. As a result, this work focuses on architectural design and feasibility instead of practical benchmarking.

D. Design-Level Outcomes

The refinement process was guided by:

- Theoretical analysis
- Insights from existing literature
- Architectural reasoning
- UML-based system modelling

The final architecture shows a careful trade-off:

- Rejecting keyword-only ranking because of its contextual limitations
- Adopting a hybrid design to balance structure with semantics

This decision was informed by theoretical analysis and aligned with both academic research and the practical needs of recruitment.

E. Observations on Constraints and Operational Factors

Throughout the development process, several operational factors were identified that were not anticipated at the start:

Constraint: Initial assumption that all resumes would be in standardized formats proved incorrect. Real-world resumes vary significantly in structure.

Action Taken: Adopted flexible NLP-based parsing instead of rigid pattern matching, improving robustness across diverse resume formats.

Constraint: Balancing computational efficiency with semantic accuracy when choosing embedding models.

Action Taken: Selected MiniLM (a lightweight Sentence-Transformer variant) that provides strong semantic understanding while maintaining reasonable processing speeds.

Constraint: Need for transparency in automated decisions became more critical than initially planned.

Action Taken: Enhanced the system to provide detailed rejection reasons and score breakdowns, making the decision process auditable.

Constraint: Database selection for handling multiple resume uploads and queries efficiently.

Action Taken: Switched from initial consideration of SQLite to PostgreSQL for better scalability and concurrent access support.

These refinements helped improve both the practical feasibility and ethical soundness of the final system design.

F. Traceability and Validation

As documented in SRS Version 2.0, we maintained a comprehensive traceability matrix mapping functional requirements (FR-1 through FR-10) and non-functional requirements (NFR-1 through NFR-9) to specific UML diagrams, modules, and validation methods. This ensures that:

- Every requirement is traceable to implementation
- Each module can be independently tested
- System behavior aligns with documented specifications

V. Conclusion

The proposed Resume Screening and Scoring system is designed to screen and rank resumes in a structured and transparent manner. Instead of depending only on keyword-based matching, the system combines rule-based filtering with semantic similarity methods. This balance allows the system to enforce strict eligibility requirements while also understanding the context of skills and experiences.

During the refinement of the SRS and the system architecture, many challenges were identified, such as resume format differences and variation in skill description. Transparency in decision-making is also important when evaluating resumes. Considering these aspects made the proposed system more realistic and feasible. The proposed design provides a structured foundation for building an effective resume screening system.

The 8-week sprint-based development approach, with clearly defined roles and deliverables, ensures systematic progress from requirements analysis to final implementation and testing. The hybrid architecture combining rule-based filtering with semantic similarity analysis addresses the identified gap in existing solutions by balancing accuracy, explainability, cost, and independence.

A. Future Work

Planned future work includes:

- Complete implementation of the system
- Comprehensive performance evaluation with real resume datasets
- Fairness and bias analysis across different demographic groups

- Support for recruiter feedback and continuous model improvement
- Extension to support additional file formats beyond PDF and DOCX
- Integration with existing HR management systems

B. AI Usage Declaration

AI tools were used to assist grammar refinement and structural organization. All architectural decisions, requirements and analysis were developed by the authors.

REFERENCES

[1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of EMNLP*, pp. 3982–3992, 2019.

[3] M. Kumar and S. Singh, "Rule-Based Expert Systems for Resume Screening," *Expert Systems with Applications*, vol. 89, pp. 112–125, 2022.

[4] R. Patel, L. Chen, and A. Williams, "Machine Learning Approaches to Automated Resume Classification," *ACM Transactions on Intelligent Systems and Technology*, vol. 13, no. 2, 2023.

Appendix

Appendix: UML Diagrams

The following UML diagrams represent the system architecture as documented in SRS Version 2.0. These diagrams depict the current state of the system design at the time of submission.

A. Use Case Diagram

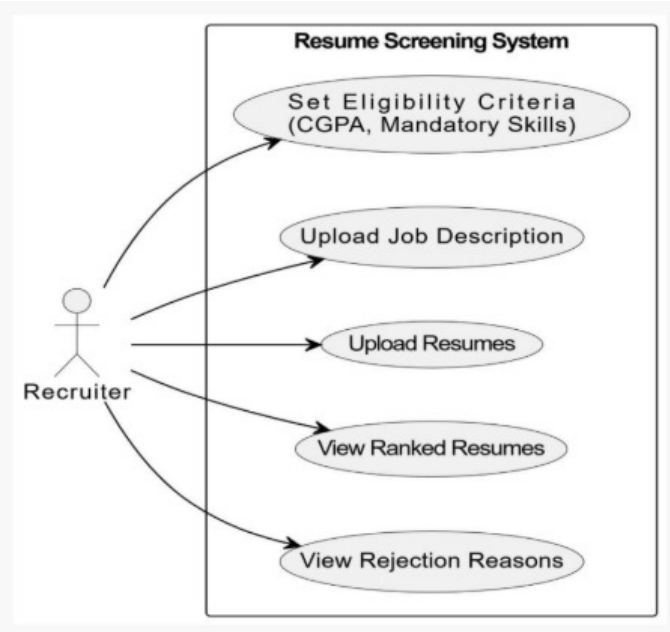

Fig. 1. Use Case Diagram showing how the HR/Recruiter interacts with the Resume Screening System to upload resumes, start screening, and view results.

The Use Case Diagram shows the recruiter can: set eligibility criteria (CGPA, mandatory skills), upload job descriptions, upload resumes, view ranked results, and view rejection reasons.

B. Sequence Diagram

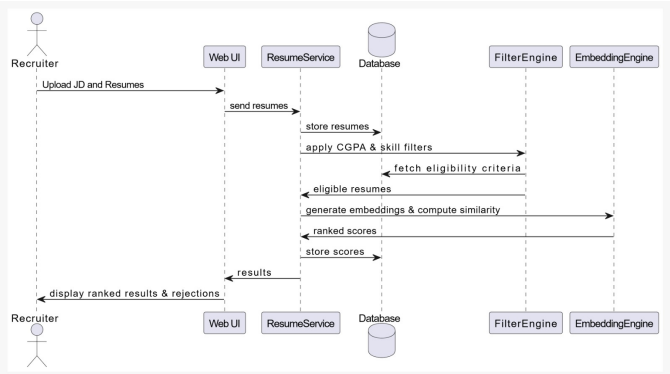

Fig. 2. Sequence Diagram showing the resume evaluation workflow from upload to result display.

The sequence diagram illustrates the complete workflow: user uploads job description and resumes through the web interface, the system extracts resume data, retrieves predefined eligibility criteria, filters resumes, computes similarity scores, ranks eligible resumes, and displays the ranked results and rejection reasons.

C. Class Diagram

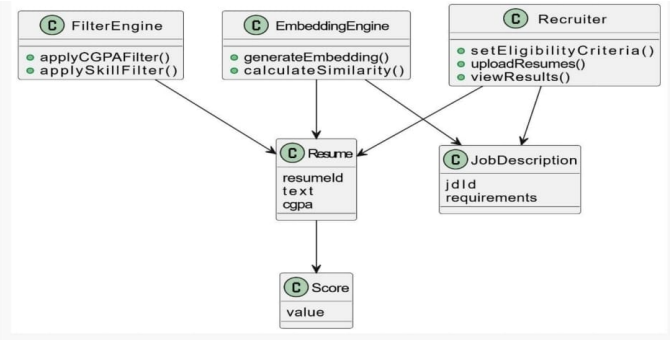

Fig. 3. Class Diagram showing key entities and their relationships in the resume screening system.

The class diagram shows the key entities: FilterEngine, EmbeddingEngine, Recruiter, Resume, JobDescription, and Score. It demonstrates clear separation of filtering and embedding logic, supporting maintainability (NFR-7).

D. Activity Diagram

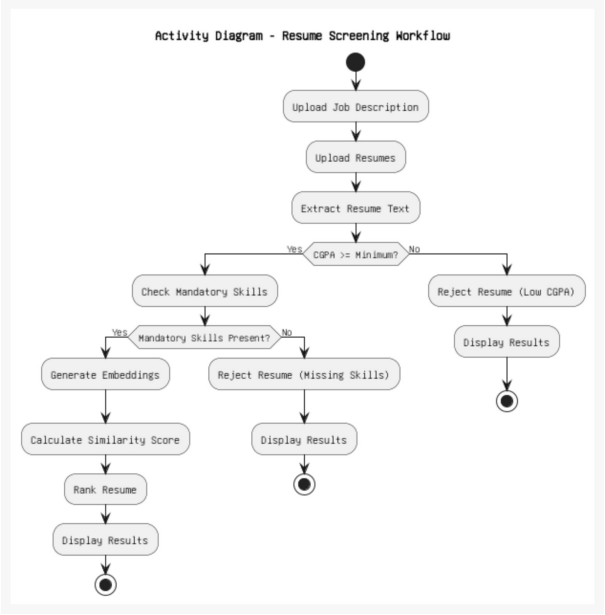

Fig. 4. Activity Diagram displaying the flow of activities from resume upload to result display.

The activity diagram shows the complete process flow: upload job description, upload resumes, extract resume text, check CGPA meets minimum, check mandatory skills present,

generate embeddings (if eligible), calculate similarity score, rank resume, and display results.

E. Component Diagram

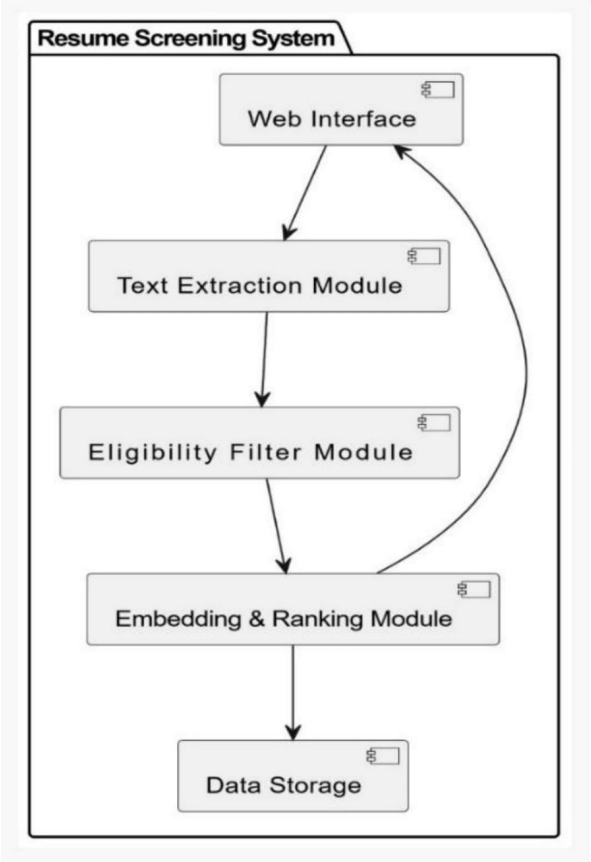

Fig. 5. Component Diagram showing the high-level software components and their interactions.

The component diagram illustrates the modular architecture: Web Interface connects to Text Extraction Module, Eligibility Filter Module, Embedding & Ranking Module, all supported by Data Storage.

F. Timing Diagram

The timing diagram illustrates state changes over time (0-50 time units): Resume File progresses from Uploaded → Parsed → Filtered → Scored → Ranked → Completed. System State transitions through Idle → Processing → ApplyingEligibility → ComputingSimilarity → Finalizing. Result Output remains Hidden until final display.

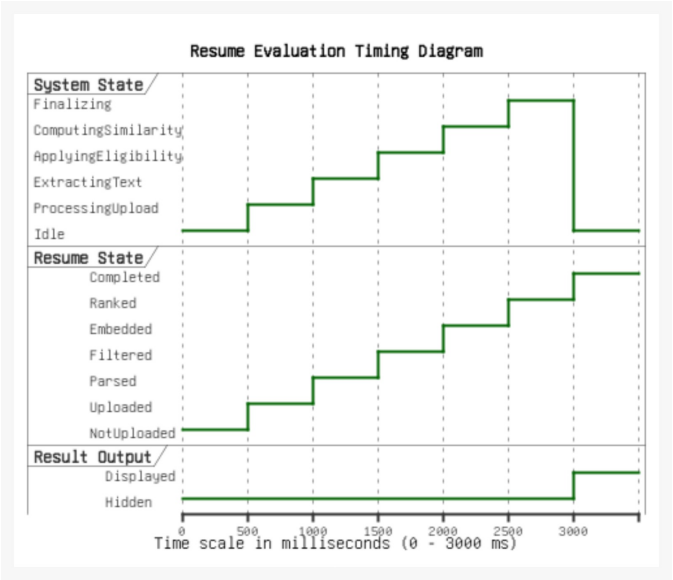

Fig. 6. Timing Diagram showing how the system and resume states change over time during the evaluation process.